

# Computing Periodic Orbits of the Lorenz System

Maximilian Hartenberger

July 22, 2026

## Abstract

This report presents a computational pipeline for discovering periodic orbits of the Lorenz system at classical parameters ( $\sigma = 10$ ,  $\rho = 28$ ,  $\beta = 8/3$ ). The approach proceeds in two stages. First, candidate orbits are identified as *near-recurrences* — close returns of a long numerically integrated trajectory — using a streaming distance-matrix method that scans narrow time windows around six target periods  $T \in \{5, 10, 20, 40, 80, 160\}$ . Second, each candidate is refined to a true periodic orbit via L-BFGS minimisation of a multiple-shooting residual, with the number of shooting segments  $N$  and the L-BFGS memory depth  $m$  varied over a  $7 \times 7$  factorial grid ( $N, m \in \{5, 10, 20, 40, 80, 160, 320\}$ ). The resulting  $\sim 29,000$  independent optimisation cases are deployed as an embarrassingly parallel workload on the Iridis X HPC cluster via SLURM job arrays. A systematic post-processing pipeline classifies every case and generates diagnostic plots (convergence curves, iteration heatmaps, and aggregate bar charts). We identify the best  $(N, m)$  pair for each target period and find that short-period orbits ( $T \leq 20$ ) converge with  $> 99\%$  success rate using as few as  $N = 5$  segments, while longer orbits require proportionally larger  $N$  and consistently benefit from the maximal L-BFGS memory  $m = 320$ . The complete pipeline — implemented in Julia using `ToySystems.jl`, `StreamingRecurrenceAnalysis.jl`, and `NKSearch.jl` — is available at <https://github.com/MaxHartenberger/Lorenz-System-Iridis>.

## 1 Introduction

Periodic orbits are fundamental invariant sets of dynamical systems. In chaotic systems such as the Lorenz equations, unstable periodic orbits (UPOs) form the *skeleton* of the attractor: every trajectory on the strange attractor can be viewed as wandering among the neighbourhoods of infinitely many UPOs [2]. Knowledge of these orbits is essential for understanding the transport, statistics, and symbolic dynamics of the system.

Computing UPOs numerically is challenging because they are unstable: a naive forward integration will diverge exponentially from the orbit. The standard remedy is to formulate the problem as a root-finding equation and solve it with a Newton-type method, using multiple shooting to manage the exponential divergence over longer periods. The convergence of such methods, however, depends critically on the quality of the initial guess.

This project addresses the initial-guess problem by first scanning a long trajectory for *near-recurrences* — pairs of times at which the flow returns close to a previously visited state. Each such near-recurrence provides an approximate period  $T_{\text{guess}}$  and a state vector  $u_{\text{rec}}$  that serve as the starting point for the refinement stage.

## 2 The Lorenz System

The Lorenz system [1] is the three-dimensional system of ordinary differential equations

$$\begin{aligned}\dot{x} &= \sigma(y - x), \\ \dot{y} &= x(\rho - z) - y, \\ \dot{z} &= xy - \beta z,\end{aligned}\tag{1}$$

with the classical parameter values  $\sigma = 10$ ,  $\rho = 28$ ,  $\beta = 8/3$ . At these values the system exhibits a chaotic strange attractor — the iconic “Lorenz butterfly.”

Let  $u(t) = (x(t), y(t), z(t)) \in \mathbb{R}^3$  denote the state at time  $t$ . The right-hand side of (1) defines a vector field  $f : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ , so that

$$\dot{u}(t) = f(u(t)). \quad (2)$$

The *flow*  $\varphi : \mathbb{R} \times \mathbb{R}^3 \rightarrow \mathbb{R}^3$  advances an initial condition  $u_0$  forward by time  $t$ :

$$\varphi(t, u_0) = u(t), \quad \dot{u} = f(u), \quad u(0) = u_0. \quad (3)$$

All numerical integrations in this work use the classic fourth-order Runge–Kutta scheme (RK4) with a fixed time step  $\Delta t = 0.01$ . The Lorenz right-hand side and the RK4 integrator are provided by the Julia package `ToySystems.jl` (<https://github.com/Davide-Lasagna-s-Lab/ToySystems.jl>), which exposes the Lorenz equations in both nonlinear and linearised forms.

### 3 Mathematical Preliminaries

Before formulating the periodic-orbit problem, we establish the mathematical objects on which the computation rests: the nonlinear flow, its linearisation, and the adjoint flow. All three are provided **matrix-free** by the Julia packages `Flows.jl` and `NKSearch.jl`; no Jacobian matrix is ever assembled.

#### 3.1 Nonlinear Flow

Let  $f \in C^k(\mathbb{R}^n)$  denote the right-hand side of a dynamical system ( $n = 3$  for Lorenz). The *flow map*  $\varphi : \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R}^n$  advances an initial condition  $u_0$  forward by time  $t$ :

$$\varphi(t, u_0) = u(t), \quad \dot{u} = f(u), \quad u(0) = u_0. \quad (4)$$

It satisfies  $\frac{\partial \varphi}{\partial t}(t, u_0) = f(\varphi(t, u_0))$ . In this work  $\varphi$  is discretised with the classical fourth-order Runge–Kutta scheme (RK4) at fixed step  $\Delta t$ .

#### 3.2 Linearised Flow

Differentiating the flow with respect to the initial condition yields the *linearised flow* (or state-transition matrix)  $\Phi(t, u_0) := \frac{\partial \varphi}{\partial u_0}(t, u_0)$ . Swapping the order of differentiation and applying the chain rule gives the variational equation

$$\frac{\partial \Phi}{\partial t} = Df(\varphi(t, u_0)) \Phi(t, u_0), \quad \Phi(0, u_0) = I. \quad (5)$$

For an initial perturbation  $v_0$ , the variation  $v(t) := \Phi(t, u_0) v_0$  satisfies

$$\dot{v} = Df(\varphi(t, u_0)) v, \quad v(0) = v_0. \quad (6)$$

For better understanding, we can look at the linearized flow as a linear approximation of the flow  $\varphi$  in the following way: Let  $v_0 = u_0 - \bar{u}_0$  be a small perturbation of the initial condition  $\bar{u}_0$ . Then using Taylor expansion we have

$$\varphi(t, \bar{u}_0) = \varphi(t, u_0) - \Phi(t, u_0) v_0 + \mathcal{O}(\|v_0\|^2).$$

The linearized flow  $\Phi$  is the first order approximation of the flow  $\varphi$  with respect to the initial condition.

$$\begin{aligned} \Phi(t, u_0)(u_0 - \bar{u}_0) &= \varphi(t, u_0) - \varphi(t, \bar{u}_0) + \mathcal{O}(\|u_0 - \bar{u}_0\|^2) \\ &\approx \varphi(t, u_0) - \varphi(t, \bar{u}_0). \end{aligned}$$

### 3.3 Adjoint Flow

The *adjoint flow*  $\Psi(t, u_0) := \Phi(t, u_0)^\top$  maps a final variation backward to the initial variation. Let  $q_T$  be a final variation at time  $T$ . Then

$$\Psi(T, u_0) q_T = q(0).$$

Define  $q(t) := \Psi(T-t, \varphi(t, u_0)) q_T$ . Using the cocycle property of the linearised flow  $\Phi(t+s, u_0) = \Phi(t, \varphi(s, u_0)) \Phi(s, u_0)$ , we have

$$\Phi(T, u_0) = \Phi(T-t, \varphi(t, u_0)) \Phi(t, u_0).$$

Taking the transpose gives the adjoint cocycle property  $\Psi(T, u_0) = \Psi(t, u_0) \Psi(T-t, \varphi(t, u_0))$ . Consequently

$$\begin{aligned} q(0) &= \Psi(T, u_0) q_T \\ &= \Psi(t, u_0) \Psi(T-t, \varphi(t, u_0)) q_T \\ &= \Psi(t, u_0) q(t), \end{aligned}$$

and thus  $q(t) = \Psi(t, u_0)^{-1} q(0) = \Phi(t, u_0)^{-\top} q(0)$ . Differentiating with respect to  $t$  and using  $\frac{d}{dt} M^{-1} = -M^{-1} \dot{M} M^{-1}$  gives

$$\begin{aligned} \frac{\partial q}{\partial t} &= -\Phi(t, u_0)^{-\top} \frac{\partial \Phi}{\partial t}(t, u_0)^\top \Phi(t, u_0)^{-\top} q(0) \\ &= -\Phi(t, u_0)^{-\top} (Df(\varphi(t, u_0)) \Phi(t, u_0))^\top \Phi(t, u_0)^{-\top} q(0) \\ &= -Df(\varphi(t, u_0))^\top q(t). \end{aligned}$$

Thus the adjoint variation  $q$  satisfies the backward-in-time ODE

$$\dot{q} = -Df(\varphi(t, u_0))^\top q, \quad q(T) = q_T. \quad (7)$$

A key duality relation holds: for any forward variation  $v(t) = \Phi(t, u_0)v_0$  and adjoint variation  $q(t)$ ,

$$\langle q_T, v(T) \rangle = \langle q_T, \Phi(T, u_0)v_0 \rangle = \langle \Phi(T, u_0)^\top q_T, v_0 \rangle = \langle \Psi(T, u_0)q_T, v_0 \rangle = \langle q(0), v_0 \rangle. \quad (8)$$

That is, the scalar product of forward and adjoint variations is conserved in time.

### 3.4 Matrix-Free Implementation in NKSearch.jl

All three flows — nonlinear  $\varphi$ , linearised  $\Phi$ , and adjoint  $\Psi$  — act **matrix-free**: only the action of  $Df$  on a vector is computed, never the full Jacobian. The RK4 integrator stores its four internal stages  $(k_1, \dots, k_4)$  in a `RAMStageCache` during the nonlinear propagation. The linearised and adjoint flows read these **same** cached stages via `TimeStepFromCache`, evaluating  $Df(x_{\text{stage}, i}) \cdot v$  at the stored base states. Because forward and adjoint propagators share the identical stage cache,

$$\langle \Phi_h \cdot v_0, q_T \rangle = \langle v_0, \Psi_h \cdot q_T \rangle \quad (9)$$

holds to machine precision — the discrete transpose relationship is exact.

Internally, `NKSearch.jl` deep-copies each flow once per shooting segment for thread safety and constructs two caches:

- **Forward cache** (`StageIterCache`): stores the nonlinear and linearised flows, manages the stage caches, and evaluates the residual  $F(z)$  via `update!`.
- **Adjoint cache** (`AdjointIterSolCache`): stores the adjoint flows and shares the stage caches, end-of-segment states, and time derivatives with the forward cache, computing  $J^\top \cdot F(z)$  via `mul!`.

This architecture is the computational engine behind the L-BFGS gradient evaluation described in section 9.

## 4 Periodic Orbits: Definition and Problem Statement

A *periodic orbit* of period  $T > 0$  is a solution  $u(t)$  of (1) satisfying

$$u(t + T) = u(t) \quad \forall t \in \mathbb{R}. \quad (10)$$

Equivalently, a point  $u_0$  lies on a periodic orbit of period  $T$  if and only if

$$\varphi(T, u_0) = u_0. \quad (11)$$

Condition (11) can be expressed as the root-finding problem

$$\tilde{F}(u_0, T) \equiv \varphi(T, u_0) - u_0 = 0. \quad (12)$$

Equation (12) is under-determined: there are four unknowns ( $u_0 \in \mathbb{R}^3$  and  $T \in \mathbb{R}$ ) but only three equations. This reflects the fact that every point on a given periodic orbit is a valid  $u_0$  for the same  $T$ . The standard resolution is to add a *phase condition* that selects a unique point on the orbit, for instance

$$\langle f(u_0), u_0 - u_{\text{ref}} \rangle = 0, \quad (13)$$

which pins  $u_0$  to a Poincaré section orthogonal to the flow and passing through a reference state  $u_{\text{ref}}$ .

For long periods  $T$ , the single-shooting formulation (12) suffers from extreme sensitivity to the initial condition because the Lorenz system is chaotic: a perturbation  $\delta u_0$  grows as  $\|\delta u(T)\| \sim e^{\lambda T} \|\delta u_0\|$ , where  $\lambda > 0$  is the largest Lyapunov exponent. This makes the root-finding problem ill-conditioned. The standard cure is *multiple shooting*, which we describe in section 8.

## 5 Near-Recurrences: Concept and Mathematical Definition

### 5.1 Intuition

On a chaotic attractor, a long trajectory  $\{u(t)\}_{t \geq 0}$  will, by the Poincaré recurrence theorem, eventually return arbitrarily close to any previously visited state. In practice, for a finite-time simulation, the trajectory will exhibit many *near-returns* — pairs of times  $t_i < t_j$  for which the states are close in phase space:

$$\|u(t_j) - u(t_i)\| \ll 1. \quad (14)$$

If the distance is small enough, the segment of trajectory between  $t_i$  and  $t_j$  is a good approximation to a periodic orbit, and  $T_{\text{guess}} \approx t_j - t_i$  is a good estimate of its period.

### 5.2 Formal Definition

Let  $u^{(0)}, u^{(1)}, u^{(2)}, \dots$  be a discrete trajectory sampled every  $\Delta t$  from a long forward integration starting from  $u^{(0)} = (1, 1, 1)^\top$ . That is,

$$u^{(k)} = \varphi^{k\Delta t}(u^{(0)}), \quad k = 0, 1, 2, \dots \quad (15)$$

A *near-recurrence* at discrete-time shift  $s$  is a pair of indices  $(k, k+s)$  such that the Euclidean distance between the two states falls below a threshold  $\varepsilon$ :

$$d(k, s) \equiv \|u^{(k+s)} - u^{(k)}\| < \varepsilon. \quad (16)$$

The approximate period associated with this recurrence is

$$T_{\text{guess}} = s \cdot \Delta t, \quad (17)$$

and the recurrence state is  $u_{\text{rec}} = u^{(k)}$  (or, equivalently, any  $u^{(k)}$  along the approximately closed loop).

In words: if the flow returns to within a distance  $\varepsilon$  of a previously visited point after  $s$  time steps, we have found a candidate for a periodic orbit of approximate period  $s\Delta t$ .

### 5.3 Why This Works for the Lorenz Attractor

The Lorenz attractor has a densely woven structure: unstable periodic orbits of many different periods coexist and organise the flow. When a trajectory comes within a small distance of itself after some time  $s\Delta t$ , it is typically shadowing an actual UPO of period close to  $s\Delta t$ . The near-recurrence provides an initial condition  $u_{\text{rec}}$  that lies within the basin of convergence of Newton’s method for that UPO. The closer the recurrence (smaller  $d$ ), the more likely the subsequent optimisation will converge.

## 6 Finding Near-Recurrences: Algorithm

### 6.1 Overview of the Pipeline

The recurrence-detection stage is implemented in the Julia script `scripts/find_recurrences.jl`. For each target period  $T_{\text{target}} \in \{5, 10, 20, 40, 80, 160\}$ , the script:

- (i) Generates a long reference trajectory by integrating the Lorenz equations with RK4 from  $u^{(0)} = (1, 1, 1)^T$ , with step  $\Delta t = 0.01$ ;
- (ii) Searches a narrow window of time shifts around  $T_{\text{target}}/\Delta t$  for all pairs of states within distance  $\varepsilon = 0.5$ ;
- (iii) Filters candidates to those whose approximate period lies within  $\pm 20\%$  of  $T_{\text{target}}$ ;
- (iv) Greedy-deduplicates the candidates by spatial separation, keeping only recurrences whose state vectors are pairwise separated by at least  $\delta_{\text{min}} = 3.0$  in phase space.

### 6.2 Search Window and Streaming Distance Matrix

For a given target period  $T_{\text{target}}$ , we expect near-recurrences at time shifts  $s \approx T_{\text{target}}/\Delta t$ . Rather than computing the full  $O(K^2)$  distance matrix for a trajectory of length  $K$ , we use a *streaming distance matrix* that only compares each state with a restricted range of future states:

$$s \in [s_{\text{centre}} - w, s_{\text{centre}} + w], \quad s_{\text{centre}} = \text{round}\left(\frac{T_{\text{target}}}{\Delta t}\right), \quad (18)$$

where the search half-width is  $w = 200$  steps (corresponding to a period range of  $\pm 2.0$  time units around  $T_{\text{target}}$ ). The streaming implementation, provided by the Julia package

`StreamingRecurrenceAnalysis.jl`

(<https://github.com/Davide-Lasagna-s-Lab/StreamingRecurrenceAnalysis.jl>), processes the trajectory in a single pass and collects all pairs  $(i, j)$  whose distance falls below the threshold  $\varepsilon$ :

$$\mathcal{R}_{\text{raw}} = \{(k, s) \mid d(k, s) < \varepsilon, s \in [s_{\text{centre}} - w, s_{\text{centre}} + w]\}. \quad (19)$$

The maximum number of “views” (distance computations) is set to  $n_{\text{views}} = 900,000$ , which limits the total computational cost of the scan.

### 6.3 Period Filtering

From the raw set  $\mathcal{R}_{\text{raw}}$ , we retain only those recurrences whose approximate period  $T_{\text{guess}} = s\Delta t$  is within a specified tolerance of the target:

$$\frac{|T_{\text{guess}} - T_{\text{target}}|}{T_{\text{target}}} \leq \tau, \quad \tau = 0.20. \quad (20)$$

This ensures that the candidate is a plausible initial guess for a periodic orbit of the desired period, rather than a harmonic or a near-recurrence at an unrelated time scale.

## 6.4 Greedy Spatial Deduplication

The remaining candidates may cluster around the same underlying periodic orbit: a single UPO can produce many near-recurrences at slightly different phases or with minor variations in approximate period. To avoid redundant computation in the optimisation stage, we apply a greedy deduplication:

- (i) Sort the filtered recurrences by ascending distance  $d$  (closest recurrences are the most promising candidates);
- (ii) Iterate through the sorted list; for each candidate, accept it only if its state vector  $u$  is at distance at least  $\delta_{\min}$  from every *already accepted* candidate:

$$\forall u_{\text{kept}} \in \mathcal{K}, \quad \|u - u_{\text{kept}}\| \geq \delta_{\min}. \quad (21)$$

The threshold  $\delta_{\min} = 3.0$  is chosen empirically; it is large enough to distinguish genuinely distinct orbits on the Lorenz attractor (characteristic size  $\sim 40$ ) while small enough not to discard valid distinct candidates.

This procedure yields a set  $\mathcal{K}$  of *distinct near-recurrence candidates*, each characterised by:

- A state vector  $u_{\text{rec}} \in \mathbb{R}^3$ ,
- An approximate period  $T_{\text{guess}}$ ,
- A discrete-time shift  $s$ ,
- A recurrence distance  $d = \|u^{(k+s)} - u^{(k)}\|$ .

## 6.5 Output

The deduplicated candidates are saved to CSV files, one per target period, under `recurrences/TXX/recurrences`. Each file contains up to 100 candidates (ranked by recurrence distance, closest first), with the columns: `rec_id`, `u1`, `u2`, `u3`, `T_guess`, `shift`, `distance`.

## 7 Summary of Key Parameters

Table 1: Parameters used in the near-recurrence detection stage.

| Symbol              | Value                        | Description                                          |
|---------------------|------------------------------|------------------------------------------------------|
| $\Delta t$          | 0.01                         | RK4 integration step                                 |
| $T_{\text{target}}$ | $\{5, 10, 20, 40, 80, 160\}$ | Target orbit periods                                 |
| $\varepsilon$       | 0.5                          | Maximum recurrence distance (Eq. 16)                 |
| $w$                 | 200 steps                    | Search half-window around $s_{\text{centre}}$        |
| $\tau$              | 0.20                         | Period tolerance (Eq. 20)                            |
| $\delta_{\min}$     | 3.0                          | Minimum spatial separation for distinctness (Eq. 21) |
| $n_{\text{views}}$  | 900,000                      | Maximum distance computations in streaming scan      |

## 8 Multiple Shooting for Periodic Orbits

### 8.1 Motivation

The single-shooting condition  $\varphi(T, u_0) = u_0$  (Eq. 11) is ill-conditioned for chaotic systems: an initial perturbation  $\delta u_0$  grows as  $\|\delta u(T)\| \sim e^{\lambda T} \|\delta u_0\|$ , where  $\lambda > 0$  is the leading Lyapunov exponent of the Lorenz attractor ( $\lambda \approx 0.91$ ). For  $T = 80$ , this amplifies errors by a factor of  $\sim 10^{31}$ , rendering the Jacobian numerically singular and the root-finding problem intractable.

*Multiple shooting* [4] circumvents this by splitting the orbit into  $N$  shorter segments, each integrated only over time  $T/N$ . The exponential amplification per segment is reduced to  $e^{\lambda T/N}$ , which can be kept  $O(1)$  by choosing  $N$  sufficiently large.

### 8.2 Formulation

Partition the period- $T$  orbit into  $N$  segments of equal duration  $T/N$ . Let  $u_i \in \mathbb{R}^3$  be the state at the beginning of segment  $i$  ( $i = 1, \dots, N$ ). The segment is integrated forward:

$$\varphi(T/N, u_i) = u_{i+1}, \quad i = 1, \dots, N, \quad (22)$$

with the periodic wrap-around convention  $u_{N+1} \equiv u_1$ .

The unknown vector collects all shooting points and the period:

$$z = \begin{pmatrix} u_1 \\ \vdots \\ u_N \\ T \end{pmatrix} \quad (23)$$

The *continuity constraints* require that the end of each segment matches the beginning of the next. This defines the **unaugmented residual**  $\tilde{F} : \mathbb{R}^{3N+1} \rightarrow \mathbb{R}^{3N}$ :

$$\tilde{F}(z) = \begin{pmatrix} \varphi(T/N, u_1) - u_2 \\ \varphi(T/N, u_2) - u_3 \\ \vdots \\ \varphi(T/N, u_N) - u_1 \end{pmatrix}. \quad (24)$$

### 8.3 Jacobian of the Residual

The Jacobian  $D\tilde{F}(z) \in \mathbb{R}^{3N \times (3N+1)}$  has a block-bidiagonal structure inherited from the segment coupling. Let  $\Phi_i := \Phi(T/N, u_i)$  denote the linearised flow (state-transition matrix) along segment  $i$ , and let  $f_i := f(\varphi(T/N, u_i))$  be the velocity at the end of the segment. Then

$$D\tilde{F}(z) = \begin{pmatrix} \Phi_1 & -I & 0 & \dots & 0 & \frac{1}{N}f_1 \\ 0 & \Phi_2 & -I & \dots & 0 & \frac{1}{N}f_2 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ -I & 0 & 0 & \dots & \Phi_N & \frac{1}{N}f_N \end{pmatrix}. \quad (25)$$

The last column arises from differentiating  $\varphi(T/N, u_i)$  with respect to  $T$ :  $\frac{\partial}{\partial T} \varphi(T/N, u_i) = \frac{1}{N} f(\varphi(T/N, u_i))$ . Each block is  $3 \times 3$ ; the full matrix has  $3N$  rows and  $3N + 1$  columns — it is one column short of being square, reflecting the time-translation symmetry of periodic orbits.

## 8.4 Phase Condition

The system  $\tilde{F}(z) = 0$  is under-determined ( $3N$  equations for  $3N + 1$  unknowns). If  $\{u_i\}_{i=1}^N$  is a solution, shifting each point along the orbit by the same time offset yields another valid solution. We fix this gauge freedom by requiring that the first shooting point lies in a Poincaré section orthogonal to the local velocity:

$$\langle u_1 - u_{\text{ref}}, f(u_{\text{ref}}) \rangle = 0, \quad (26)$$

where  $u_{\text{ref}}$  is a reference point. We take  $u_{\text{ref}} = u_{\text{rec}}$ , the recurrence state, so that the phase condition is automatically satisfied at the initial guess:  $\langle u_{\text{rec}} - u_{\text{rec}}, f(u_{\text{rec}}) \rangle = 0$ . Appending this scalar equation to the residual and its gradient row to the Jacobian gives the **augmented** system  $F : \mathbb{R}^{3N+1} \rightarrow \mathbb{R}^{3N+1}$ :

$$F(z) = \begin{pmatrix} \varphi(T/N, u_1) - u_2 \\ \vdots \\ \varphi(T/N, u_N) - u_1 \\ 0 \end{pmatrix}, \quad J(z) := DF(z) = \begin{pmatrix} \Phi_1 & -I & 0 & \dots & 0 & \frac{1}{N}f_1 \\ 0 & \Phi_2 & -I & \dots & 0 & \frac{1}{N}f_2 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ -I & 0 & 0 & \dots & \Phi_N & \frac{1}{N}f_N \\ f(u_{\text{ref}})^\top & 0 & 0 & \dots & 0 & 0 \end{pmatrix}. \quad (27)$$

The phase-condition row  $f(u_{\text{ref}})^\top$  is constant (independent of  $z$ ) because we fix the reference at the start of the optimisation. The zero appended to  $\tilde{F}$  ensures matching dimensions; the phase condition is identically satisfied at the initial guess by construction, so its residual entry remains zero throughout.

## 8.5 Least-Squares Reformulation

Rather than solving  $F(z) = 0$  directly with Newton's method (which requires  $J(z)$  to be well-conditioned and involves solving a  $(3N + 1) \times (3N + 1)$  linear system at each iteration), we minimise the scalar cost function

$$\phi(z) = \frac{1}{2} \|F(z)\|^2, \quad (28)$$

whose global minimum  $\phi = 0$  corresponds to an exact periodic orbit. The gradient of  $\phi$  is

$$\nabla \phi(z) = J(z)^\top F(z), \quad (29)$$

which can be computed matrix-free using the adjoint flow (section 9.2). This least-squares formulation is the starting point for the L-BFGS optimisation described in the next section.

# 9 L-BFGS Optimisation

## 9.1 The L-BFGS Method

The Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) method [3, 7] is a quasi-Newton algorithm for minimising  $\phi(z) = \frac{1}{2} \|F(z)\|^2$ . The true Hessian of  $\phi$  is

$$\nabla^2 \phi(z) = J(z)^\top J(z) + \sum_{i=1}^{3N+1} F_i(z) \nabla^2 F_i(z), \quad (30)$$

where  $J(z) = DF(z)$  is the augmented Jacobian (Eq. 27). Near a solution ( $F_i \approx 0$ ), the second term is negligible and  $\nabla^2 \phi \approx J^\top J$ , the Gauss–Newton approximation. L-BFGS builds an approximation  $H_k \approx (\nabla^2 \phi(z_k))^{-1}$  to the inverse Hessian using only the past  $m$  gradient



differences  $s_j = z_{j+1} - z_j$  and  $y_j = \nabla\phi(z_{j+1}) - \nabla\phi(z_j)$ . At each iteration  $k$ , the search direction is computed via the **two-loop recursion** (Algorithm 2):

$$d_k = -H_k \nabla\phi(z_k), \quad (31)$$

and a backtracking line search determines the step length  $\lambda$  satisfying the simple decrease condition  $\phi(z_k + \lambda d_k) < \phi(z_k)$ . The state is then updated:  $z_{k+1} = z_k + \lambda d_k$ .

The gradient  $\nabla\phi(z) = J(z)^\top F(z)$  is evaluated **matrix-free** using the discrete adjoint (see §9.2). The key advantage of L-BFGS for this problem is that it requires only gradient information — not the full Hessian — and the limited-memory form stores only  $O(m)$  vectors rather than an  $O((3N+1)^2)$  matrix. This is essential when  $N$  is large (up to 320 segments, giving 961 unknowns). The memory parameter  $m$  controls how many past curvature pairs are retained; larger  $m$  gives a richer Hessian approximation at the cost of greater storage.

**Implementation in NKSearch.jl.** The L-BFGS method is implemented in `NKSearch.jl/src/search_lbf`. The main function `_search_lbfgs_opt!` receives both the forward cache `fwd_cache` (`StageIterCache`) and the adjoint cache `adj_cache` (`AdjointIterSolCache`), pre-built by the dispatch layer. The two caches share stage caches, end-of-segment states, and time derivatives; `update!` populates them, and the adjoint `mul!` reads them to compute  $J^\top \cdot F(z)$ . The state is stored in an `OptLBFGSCache` struct containing circular buffers `s_history`, `y_history` of `MVectors` for the step–gradient pairs, the current residual and gradient, the previous iterate, the effective history length  $n_{\text{hist}}$  (capped at  $m$ ), and the search direction. The line search (`linesearch_opt_lbfgs`) backtracks from  $\lambda = 1$ , reducing  $\lambda$  by  $\rho = 0.5$  at each rejection; invalid evaluation points (from integration failures) are caught via try-catch and treated as failures.

---

**Algorithm 1** L-BFGS for  $\min_z \frac{1}{2} \|F(z)\|^2$

---

```

1: Input: initial guess  $z_0$ , memory size  $m$ , tolerances  $\varepsilon_e, \varepsilon_d$ 
2: Allocate history buffers  $\{s_i\}_{i=1}^m, \{y_i\}_{i=1}^m$ , set  $n_{\text{hist}} \leftarrow 0$ 
3: Compute  $F_0 \leftarrow F(z_0)$ ,  $\nabla\phi_0 \leftarrow J(z_0)^\top F_0$ ,  $e \leftarrow \|F_0\|$ 
4: for  $k = 0, 1, 2, \dots$  until convergence do
5:   if  $e < \varepsilon_e$  then
6:     break
7:   end if
8:   Compute search direction  $d_k$  via two-loop recursion (Alg. 2)
9:    $z_{\text{prev}} \leftarrow z_k$ ,  $\nabla\phi_{\text{prev}} \leftarrow \nabla\phi_k$ 
10:  Line search: find  $\lambda$  s.t.  $\phi(z_k + \lambda d_k) < \phi(z_k)$ 
11:   $z_{k+1} \leftarrow z_k + \lambda d_k$ 
12:  Compute  $\nabla\phi_{k+1} \leftarrow J(z_{k+1})^\top F(z_{k+1})$ ,  $e \leftarrow \|F(z_{k+1})\|$ 
13:  Update history with  $(s = z_{k+1} - z_k, y = \nabla\phi_{k+1} - \nabla\phi_k)$ 
14:  if  $\|s\| < 10^{-14}$  or  $\|y\| < 10^{-14}$  or  $\langle s, y \rangle \leq 0$  then
15:    skip
16:  end if
17:   $\text{idx} \leftarrow (k \bmod m) + 1$ ;  $s_{\text{idx}} \leftarrow s$ ;  $y_{\text{idx}} \leftarrow y$ ;  $n_{\text{hist}} \leftarrow \min(n_{\text{hist}} + 1, m)$ 
18:  if  $\|\lambda d_k\| < \varepsilon_d$  then
19:    break
20:  end if
21: end for
22: return  $z_k$ 

```

---

---

**Algorithm 2** L-BFGS two-loop recursion — computes  $d = -H_k^{-1}\nabla\phi$

---

```

1: Input: gradient  $\nabla\phi$ , history  $\{(s_i, y_i)\}_{i=1}^n$ , step  $k$ , memory  $m$ 
2:  $n \leftarrow \min(k, m)$  ▷ effective history length
3:  $q \leftarrow -\nabla\phi$ 
4: for  $i = n, n-1, \dots, 1$  do ▷ first loop (backward)
5:    $\rho_i \leftarrow 1/\langle y_i, s_i \rangle$ 
6:    $\alpha_i \leftarrow \rho_i \langle s_i, q \rangle$ 
7:    $q \leftarrow q - \alpha_i y_i$ 
8: end for
9:  $\gamma \leftarrow \begin{cases} \langle s_n, y_n \rangle / \langle y_n, y_n \rangle & \text{if } n > 0 \\ 1 & \text{otherwise} \end{cases}$  ▷ initial scaling
10:  $q \leftarrow \gamma q$ 
11: for  $i = 1, 2, \dots, n$  do ▷ second loop (forward)
12:    $\beta \leftarrow \rho_i \langle y_i, q \rangle$ 
13:    $q \leftarrow q + (\alpha_i - \beta) s_i$ 
14: end for
15: return  $d \leftarrow q$ 

```

---

## 9.2 Gradient via Adjoint Integration

Computing the gradient  $\nabla\phi(z) = J(z)^\top F(z)$  requires the transpose Jacobian. Thanks to the block-bidiagonal structure of  $J(z)$  (Eq. 27), the product  $J^\top F$  can be evaluated **matrix-free** by integrating the adjoint equations backward along each segment. The forward propagation (**update!**) populates the RK4 stage caches; the adjoint propagation (**mul!**) reads these same caches to compute  $J^\top \cdot F$ , using the adjoint ODE  $\dot{q} = -Df(\varphi(t, u_i))^\top q$  (Eq. 7) backward from  $t = T/N$  to  $t = 0$  on each segment. Because the forward and adjoint propagators share identical stage caches (see §3.4), the discrete duality  $\langle Jv, w \rangle = \langle v, J^\top w \rangle$  holds to machine precision.

The implementation in `NKSearch.jl` stores the L-BFGS state in an `OptLBFGSCache` struct containing:

- `s_history`, `y_history` — circular buffers of `MVectors` for the step–gradient pairs;
- `Fz_curr`, `grad_curr`, `grad_prev` — current and previous residual and gradient;
- `z_prev` — previous iterate;
- `n_hist` — effective history length (capped at  $m$ );
- `direction` — search direction, overwritten each iteration.

The line search is a simple backtracking procedure: starting from  $\lambda = 1$ , it evaluates  $\phi(z + \lambda d)$  by temporarily stepping  $z$ , calling **update!** to compute the residual, and restoring  $z$ . If the trial point has a lower objective value, the step is accepted; otherwise  $\lambda$  is reduced by a factor  $\rho = 0.5$ . Invalid evaluation points (e.g. from integration failures) are caught and treated as failures, triggering further backtracking.

## 9.3 Initial Guess from Near-Recurrences

For each near-recurrence candidate  $(u_{\text{rec}}, T_{\text{guess}})$ , the initial multiple-shooting state  $z^{(0)}$  is constructed as:

$$u_i^{(0)} = \varphi((i-1)T_{\text{guess}}/N, u_{\text{rec}}), \quad i = 1, \dots, N, \quad (32)$$

with  $T^{(0)} = T_{\text{guess}}$ . In words: the recurrence state is propagated forward in  $N-1$  equal time increments to seed all  $N$  shooting points along the approximate orbit. If the near-recurrence is

sufficiently close to a true periodic orbit, this initial guess lies within the basin of convergence of L-BFGS.

## 9.4 Parameter Sweep

The quality of convergence depends on two numerical parameters:

- $N$  (number of shooting segments): controls the conditioning of the problem. Too few segments  $\Rightarrow$  the Jacobian is ill-conditioned; too many  $\Rightarrow$  the search space is unnecessarily large.
- $m$  (L-BFGS memory depth): controls how much curvature history informs the Hessian approximation. Small  $m$  may lead to slow convergence; large  $m$  increases memory but may capture more accurate local curvature.

There is no *a priori* optimal choice of  $(N, m)$  — it depends on the specific orbit. We therefore run a **full factorial sweep** over a  $7 \times 7$  grid:

$$N \in \{5, 10, 20, 40, 80, 160, 320\}, \quad m \in \{5, 10, 20, 40, 80, 160, 320\}. \quad (33)$$

For longer target periods ( $T \geq 40$ ), small values of  $N$  and  $m$  are omitted because they are known *a priori* to diverge (insufficient segment resolution for the longer orbit). In more detail, the parameter sweep is:

- $T = 5$ : all  $7 \times 7$  combinations of  $(N, m)$ ;
- $T = 10$ : all  $7 \times 7$  combinations of  $(N, m)$ ;
- $T = 20$ : all  $7 \times 7$  combinations of  $(N, m)$ ;
- $T = 40$ :  $N \in \{10, 20, 40, 80, 160, 320\}$ ,  $m \in \{10, 20, 40, 80, 160, 320\}$ ;
- $T = 80$ :  $N \in \{20, 40, 80, 160, 320\}$ ,  $m \in \{20, 40, 80, 160, 320\}$ ;
- $T = 160$ :  $N \in \{40, 80, 160, 320\}$ ,  $m \in \{40, 80, 160, 320\}$ .

## 9.5 Convergence and Monitoring

The optimisation is declared **converged** when  $\|F(z)\| \leq 10^{-8}$ . A maximum of  $10^7$  iterations is allowed; runs reaching this limit without satisfying the tolerance are marked as **hit\_maxiter**. Runs in which the residual norm diverges to NaN are classified as **crashed**. At each iteration, the following quantities are recorded to a per-case CSV file:

$$\text{iter}, \|F\|, \|\nabla\phi\|, \lambda, T_{\text{curr}},$$

where  $\lambda$  is the line-search step length and  $T_{\text{curr}}$  is the current estimate of the period. When convergence is achieved, the converged trajectory is integrated and saved as a full phase-space time series for visualisation.

# 10 High-Performance Computing Deployment

## 10.1 Embarrassingly Parallel Structure

The parameter sweep defines a four-dimensional grid over  $(T, \text{rec\_id}, N, m)$ . Every combination is an **independent** optimisation problem: no communication or synchronisation is required between cases. The total number of cases is

$$\underbrace{6}_{T \text{ values}} \times \underbrace{\leq 100}_{\text{rec\_id per } T} \times \underbrace{\leq 7}_{N \text{ values}} \times \underbrace{\leq 7}_{m \text{ values}} \approx 29,400. \quad (34)$$

This is a classic *embarrassingly parallel* workload, ideally suited to a high-throughput computing cluster.

## 10.2 Iridis X Cluster and SLURM Job Arrays

All computations were performed on the **Iridis X** HPC cluster at the University of Southampton (see <https://soton.ac.sharepoint.com/teams/HPCCommunityWiki> on how to use it). The full source code, SLURM scripts, analysis tools, and results are available in the project repository: <https://github.com/MaxHartenberger/Lorenz-System-Iridis>.

The workload is managed via **SLURM job arrays**: a single job script (`first_sweep.slurm`) is submitted with an array index, and SLURM launches one task per index, each executing the Julia worker script `hpc_worker.jl` with the parameters for a single  $(T, \text{rec\_id}, N, m)$  case.

The task list is pre-generated by `generate_task_list.jl` into a flat text file `tasks.txt`, with one line per case:

```
T rec_id N m
```

Because Iridis X enforces a maximum array index of 1001, the task file is split into chunks of at most 1000 lines each:

```
split -d -l 1000 tasks.txt chunk_  
sbatch --array=1-1000 first_sweep.slurm chunk_00  
sbatch --array=1-1000 first_sweep.slurm chunk_01  
...
```

Each array task reads its assigned line from the chunk file and launches `hpc_worker.jl` with the four parameters.

## 10.3 Resource Configuration

Each individual optimisation case is lightweight:

- **CPU**: 1 core per task (single-threaded Julia);
- **Memory**:  $\sim 3.7$  GB per task;
- **Walltime**: 4 hours per task (most cases converge within minutes to a few hours; the limit safeguards against runaway cases);
- **Partition**: `amd_student` (student-accessible nodes).

Output files are written to the `outputs/` directory at the repository root. The default output path of `hpc_worker.jl` is `output/`, but production sweeps were launched with `-output-dir outputs` to place results in a single, self-contained tree. Because a full sweep produces  $\sim 29,000$  CSV files, care was taken to stay within the filesystem inode quotas (the scratch filesystem, with 500,000 inodes, is available as an alternative for larger-scale runs).

## 10.4 Output Structure

The results directory mirrors the parameter hierarchy:

```
outputs/  
|-- T05/data_lbfgs/rec001/iteration/  
|   |-- N05_m05.csv  
|   |-- N05_m10.csv  
|   |-- ... (up to 49 files per rec_id)
```

```
|  |-- trajectory/
|      |-- N05_m05_trajectory.csv    (only if converged)
|-- T10/data_lbfgs/ ...
|-- ...
```

Each iteration CSV records the per-iteration history (residual norm, gradient norm, damping parameter, current period estimate) and concludes with a status marker (`# converged`, `# crashed` or `# hit_maxiter`). Converged cases additionally produce a trajectory CSV containing the full phase-space orbit sampled at every  $\Delta t = 0.01$ .

## 11 Results Analysis Pipeline

The HPC sweep produces approximately 29,000 per-iteration CSV files across 6 target periods, up to 100 recurrence candidates each, and 49  $(N, m)$  combinations per candidate. A Python post-processing pipeline (`analysis/analyze_hpc_results.py`) reads every CSV, classifies each case, and aggregates the results into summary tables. A companion script (`analysis/plot_results.py`) generates publication-quality diagnostic plots.

### 11.1 Case Classification

Each  $(T, \text{rec\_id}, N, m)$  case is assigned one of the following statuses based on its iteration CSV:

Table 2: Classification logic for each optimisation case. The script reads only the first  $\sim 5$  data rows (NaN scan) and the last two lines (comment marker and final data row), making it efficient for  $\sim 29,000$  files.

| Status      | Criterion                                                                                                                      |
|-------------|--------------------------------------------------------------------------------------------------------------------------------|
| converged   | Final comment = <code># converged</code> and $\ f\  \leq 10^{-8}$ on the last data row.                                        |
| diverged    | NaN appears in the $\ f\ $ column within the first few data rows.                                                              |
| hit_maxiter | Final comment = <code># hit_maxiter</code> and iteration count $\geq 10^7$ .                                                   |
| incomplete  | Optimisation stopped before convergence but did not reach <code>maxiter</code> (includes runs killed by the SLURM time limit). |
| crashed     | Final comment = <code># crashed</code> (exception caught by the Julia worker).                                                 |
| no_data     | The CSV file does not exist (the worker never reached this case before the job was terminated).                                |

### 11.2 Convergence Summary by Period

Table 3 reports the classification counts for each target period. The total number of cases per  $T$  varies because (i) the recurrence finder produces different numbers of distinct candidates for each period, and (ii) for  $T \geq 40$ , small values of  $N$  and  $m$  were excluded *a priori* to avoid known divergence.

The convergence rate is excellent for short-period orbits ( $> 99\%$  for  $T \leq 20$ ) but degrades substantially for longer periods. Two factors contribute to this degradation. First, longer orbits are intrinsically harder to converge: the near-recurrence candidates tend to be farther from true periodic orbits (larger  $d$ ), and the multiple-shooting Jacobian becomes less well-conditioned

Table 3: Status summary by target period. The `no_data` cases for  $T \geq 40$  are predominantly due to the 12-hour SLURM walltime limit: the per-recurrence worker could not finish all 49  $(N, m)$  combinations before being terminated.

| <b>T</b> | <b>Converged</b> | <b>No data</b> | <b>Crashed</b> | <b>Diverged</b> | <b>Conv. rate</b> |
|----------|------------------|----------------|----------------|-----------------|-------------------|
| 5        | 4,109            | 0              | 7              | 0               | 99.8%             |
| 10       | 4,839            | 9              | 3              | 0               | 99.8%             |
| 20       | 4,824            | 14             | 6              | 7               | 99.4%             |
| 40       | 3,020            | 1,353          | 527            | 0               | 61.6%             |
| 80       | 2,022            | 2,360          | 268            | 5               | 43.4%             |
| 160      | 859              | 3,737          | 299            | 5               | 17.5%             |

even with many segments. Second, and more practically, the 12-hour SLURM time limit on the per-recurrence worker meant that many  $(N, m)$  combinations for  $T \geq 40$  were never attempted (`no_data`), and some running optimisations were killed mid-iteration (`crashed`). The true convergence rate for  $T = 160$  — if all combinations had been given sufficient time — is likely higher than the 17.5% reported here, though the intrinsic difficulty of the longest orbits remains the dominant factor.

## 12 Diagnostic Plots

The plotting script `analysis/plot_results.py` produces diagnostic figures, all saved as both PDF and PNG in `analysis/plots/`. Only recurrences with at least one converged  $(N, m)$  combination contribute data. We present a selection of the most informative plots below; the complete set (over 500 per-recurrence figures plus aggregate charts) is available in the project repository.

### 12.1 Convergence Curves (Fig. 1)

For each recurrence, a multi-panel figure is produced with one subplot per  $N$  (limited to those  $N$  where at least one  $m$  converged). Each subplot shows  $\|F(z)\|$  versus iteration number on log-log axes for all converged  $m$  values. These curves reveal the convergence behaviour of the L-BFGS algorithm: the slope indicates the convergence rate, and plateaus or oscillations may indicate ill-conditioning or line-search difficulties. The final residual level ( $\sim 10^{-8}$ – $10^{-9}$ ) confirms that the convergence criterion is satisfied with a margin. Figure 1 shows a representative example for a short-period recurrence ( $T_{\text{target}} = 5$ , `rec_id=1`).



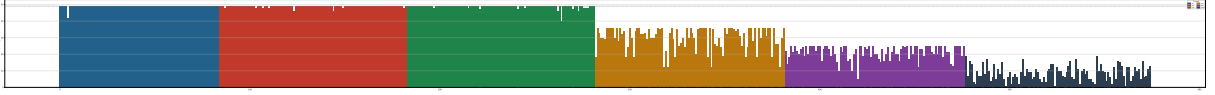


Figure 3: Convergence count per recurrence (out of 49 max). Most short-period recurrences achieve the maximum 49/49, while long-period recurrences show a broader spread.

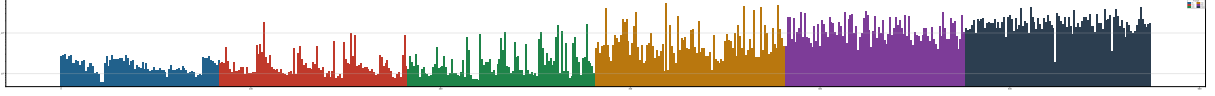


Figure 4: Iterations of the slowest converged  $(N, m)$  combination for each recurrence. This indicates the worst-case computational cost for a given orbit.

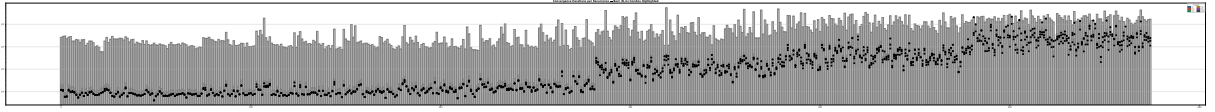


Figure 5: Slowest converged combo per recurrence (grey bars), with the three fastest  $(N, m)$  combinations overlaid as coloured scatter markers. This reveals whether the fastest combinations cluster around particular parameter values.

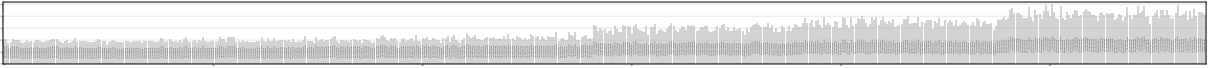


Figure 6: Fastest converged  $(N, m)$  combination per recurrence. Each bar is annotated with the recurrence ID, actual converged period  $T$ , and the winning  $(N, m)$  pair.

## 12.4 Aggregate Heatmaps per Period

To summarise convergence behaviour across all recurrences for a given target period, we produce a two-panel figure for each  $T$ : the left panel shows the median number of L-BFGS iterations to convergence for each  $(N, m)$  pair (logarithmic colour scale), and the right panel shows the fraction of recurrences that converged for that pair (green = high success rate, red = low). Figures 7–12 present the six aggregate heatmaps.



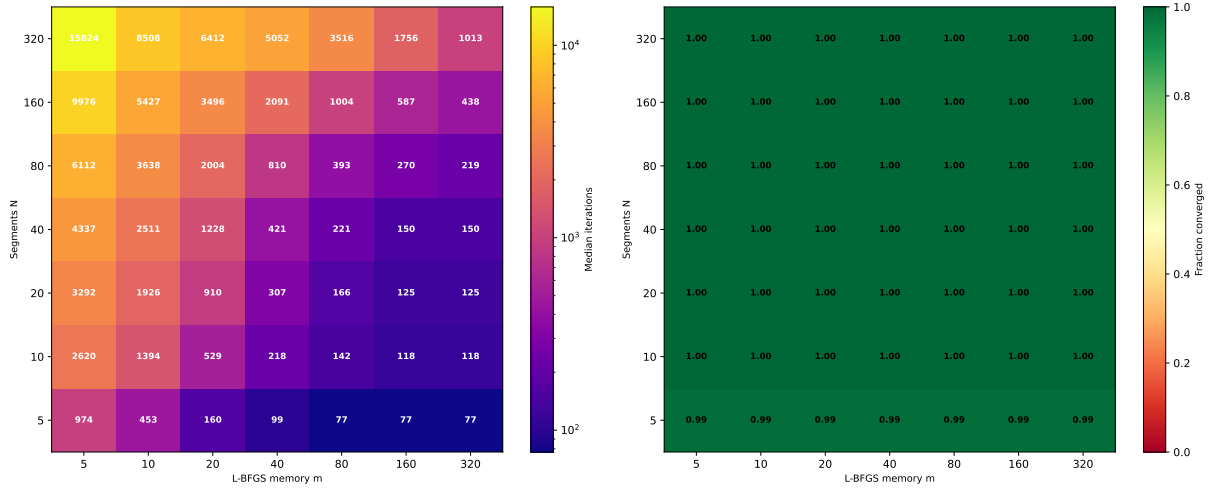


Figure 7: Aggregate heatmap for  $T = 5$ : median iterations (left) and convergence success rate (right) across all 84 recurrences.

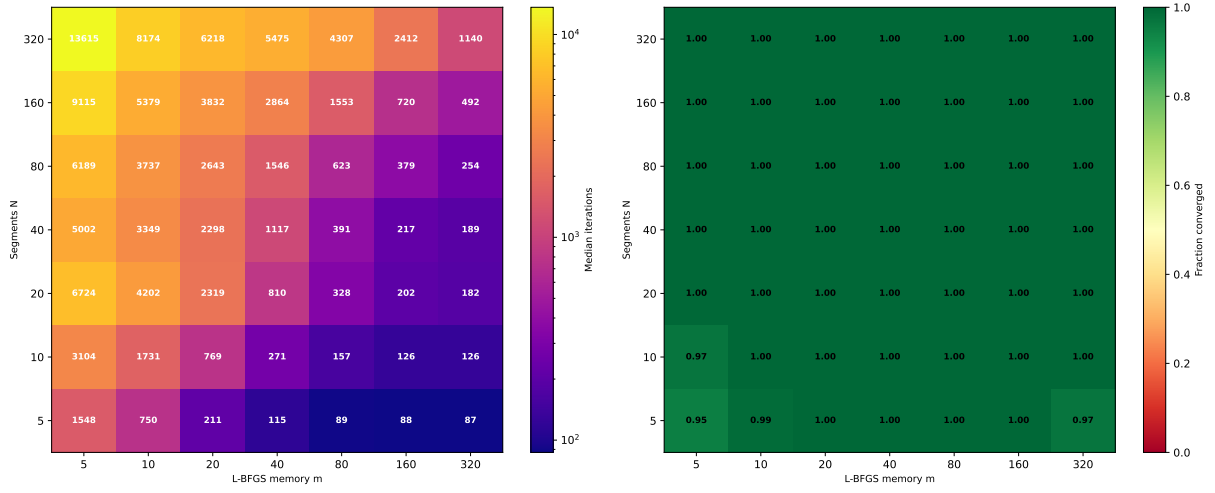


Figure 8: Aggregate heatmap for  $T = 10$  (99 recurrences).

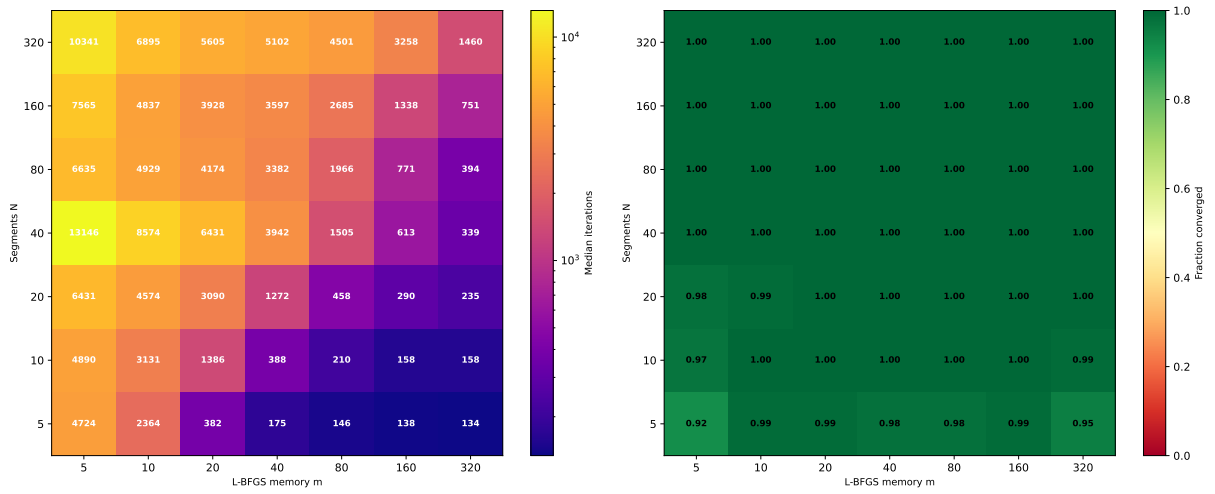


Figure 9: Aggregate heatmap for  $T = 20$  (99 recurrences). The optimal  $N$  remains at 5, but  $m = 320$  begins to overtake  $m = 160$ .

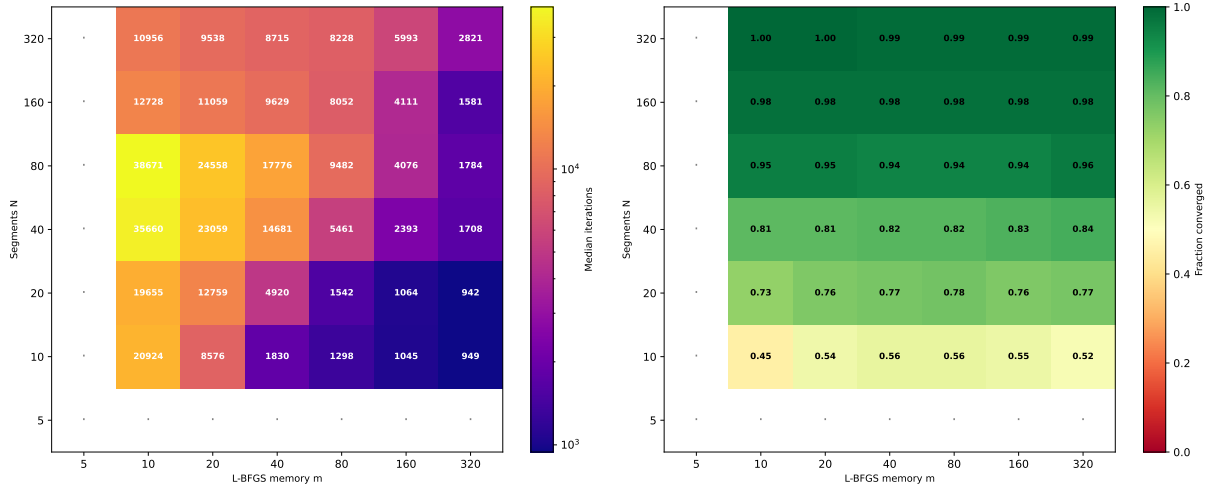


Figure 10: Aggregate heatmap for  $T = 40$  (100 recurrences). The optimal  $N$  shifts to 20; small  $N$  and  $m$  values (excluded *a priori*) appear blank.

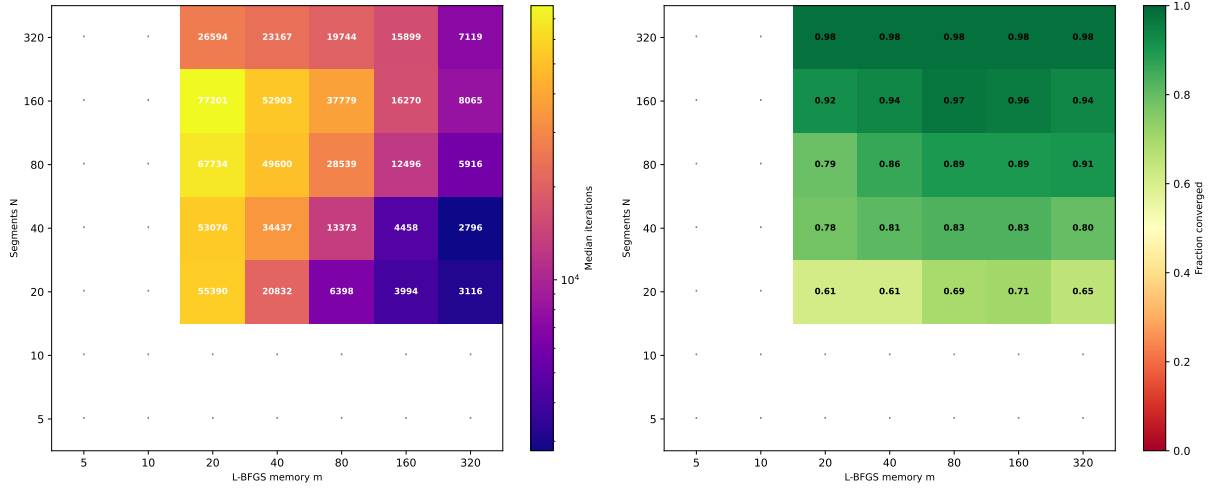


Figure 11: Aggregate heatmap for  $T = 80$  (95 recurrences). Optimal  $N$  shifts to 40;  $m = 320$  dominates the high-success region.

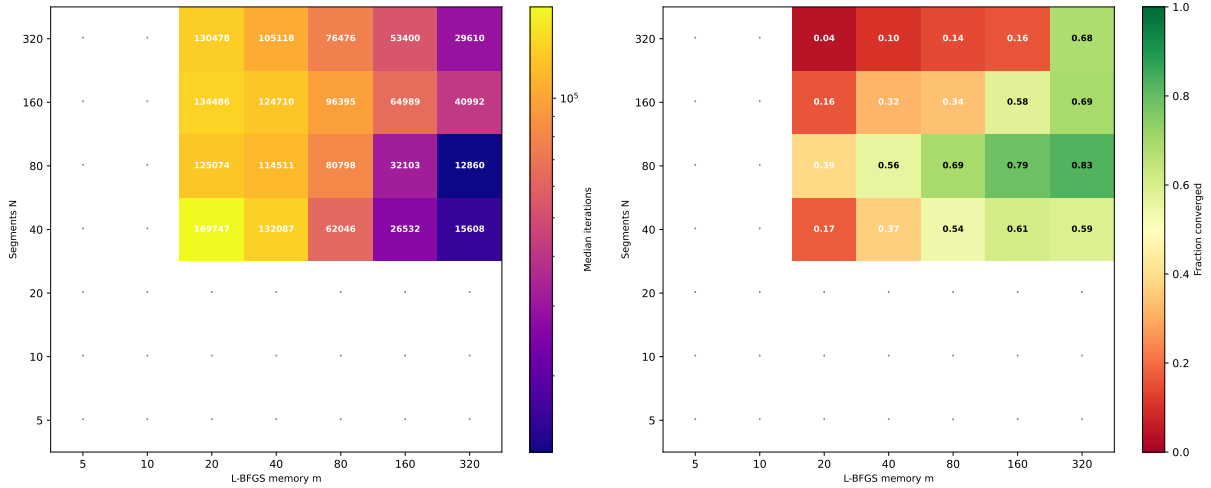


Figure 12: Aggregate heatmap for  $T = 160$  (98 recurrences). Optimal  $N = 80$ ; the convergence rate drops substantially, reflecting the intrinsic difficulty of long-period orbits.

### 13 Best Parameter Pairs per Period

A key practical question is: *which  $(N, m)$  combination is most likely to converge fastest for a given target period?* To answer this, we tabulate, for each  $T$ , the  $(N, m)$  pair that delivers the fewest iterations on the largest number of recurrences (the “winner”), together with the runner-up. Table 4 presents these results.

Table 4: Best  $(N, m)$  pair per target period. “Wins” counts how many recurrences (out of those with at least one converged combo) are converged fastest by the given pair. The win fraction is  $\text{wins} / \text{total\_recs\_with\_converged}$ .

| $T$ | Best $N$ | Best $m$ | Wins | Total | Fraction | Runner-up $N$ | Runner-up $m$ |
|-----|----------|----------|------|-------|----------|---------------|---------------|
| 5   | 5        | 160      | 73   | 84    | 86.9%    | 5             | 80            |
| 10  | 5        | 160      | 82   | 99    | 82.8%    | 5             | 320           |
| 20  | 5        | 320      | 68   | 99    | 68.7%    | 5             | 160           |
| 40  | 20       | 320      | 35   | 100   | 35.0%    | 160           | 320           |
| 80  | 40       | 320      | 46   | 95    | 48.4%    | 20            | 320           |
| 160 | 80       | 320      | 49   | 98    | 50.0%    | 40            | 320           |

Several clear trends emerge from Table 4:

- (i) **Short periods favour few segments.** For  $T \leq 20$ , the optimal number of shooting segments is  $N = 5$  — the smallest value in the sweep. This is remarkable: a single near-recurrence, seeded with just 5 shooting points, converges robustly to a periodic orbit for over 99% of candidates. The exponential amplification over  $T/N \leq 4$  time units is evidently manageable for the Lorenz attractor at these periods.
- (ii) **Optimal  $N$  scales with  $T$ .** As the period increases, the best  $N$  grows from 5 ( $T = 5$ ) to 80 ( $T = 160$ ), keeping the segment duration  $T/N$  in the range 1–2 time units. This is consistent with the multiple-shooting motivation: the segment integration time must be kept below the Lyapunov time  $1/\lambda \approx 1.1$  for the Jacobian to remain well-conditioned.
- (iii) **Large L-BFGS memory is universally beneficial.**  $m = 320$  (the maximum value in the sweep) is either the best or the runner-up memory size for *every* target period.

This suggests that the Hessian approximation benefits from retaining as much curvature history as possible, and that the storage cost of  $m = 320$  pairs ( $\sim 320 \times 961$  floating-point numbers  $\approx 2.5$  MB) is negligible relative to the computational cost of the line-search and flow integrations.

- (iv) **Win fractions decrease with  $T$ .** For  $T = 5$ , a single  $(N, m) = (5, 160)$  pair is fastest for 87% of recurrences. For  $T = 160$ , the best pair  $(80, 320)$  wins only 50% of the time. This indicates that long-period orbits are more heterogeneous: different orbits benefit from different  $(N, m)$  combinations, and no single choice dominates.

## 14 Summary and Discussion

We have presented a complete computational pipeline for discovering periodic orbits of the Lorenz system. The approach combines (i) a streaming distance-matrix method to detect near-recurrence candidates from a long trajectory, (ii) a multiple-shooting formulation that renders the root-finding problem well-conditioned, (iii) L-BFGS optimisation to refine each candidate to machine precision, and (iv) an embarrassingly parallel HPC deployment that scales to  $\sim 29,000$  independent cases on the Iridis X cluster.

The key findings are:

- Near-recurrence detection with a  $\pm 20\%$  period tolerance and spatial deduplication ( $\delta_{\min} = 3.0$ ) reliably produces  $\sim 100$  distinct candidates per target period.
- Short-period orbits ( $T \leq 20$ ) converge with  $> 99\%$  success rate using as few as  $N = 5$  shooting segments, demonstrating the quality of the near-recurrence initial guesses.
- For longer periods, the optimal number of segments scales as  $N \propto T$ , keeping the per-segment integration time below the Lyapunov time. The largest L-BFGS memory ( $m = 320$ ) is consistently the best or near-best choice.

The complete dataset — including per-iteration histories, converged trajectories, classification tables, and diagnostic plots — is available in the project repository. The pipeline is modular and can be adapted to other dynamical systems by replacing the Lorenz-specific flow and linearisation operators.

## 15 Outlook and Future Work

The present work demonstrates a robust, scalable pipeline for computing periodic orbits of the Lorenz system. Several natural extensions present themselves.

### 15.1 Application to Other Fluid-Dynamical Systems

The pipeline is modular: the recurrence finder, the multiple-shooting formulation, and the L-BFGS optimiser are agnostic to the specific dynamical system. Only the flow operators  $\varphi$ ,  $\Phi$ , and  $\Psi$  need to be replaced. A natural next target is the **Open Kolmogorov Flow** (OKF) [6], a two-dimensional, doubly-periodic, spatially-extended Navier–Stokes configuration that exhibits a rich hierarchy of exact coherent structures including travelling waves, periodic orbits, and relative periodic orbits.

## 15.2 Computational-Time Callbacks and Algorithm Comparison

A natural question is how L-BFGS compares with other solvers available in NKSearch.jl, particularly the **Newton–GMRES Hookstep** (trust-region) method [4]. The hookstep method solves  $J(z)\delta z = -F(z)$  matrix-free at each Newton iteration: GMRES approximately solves the linear system, and the step is constrained to lie within a trust region of radius  $\Delta$ . Unlike L-BFGS, the hookstep method does not require adjoint integration (only the forward linearised flow  $\Phi$ ), making each iteration computationally cheaper. However, it typically requires more iterations to converge because the trust-region constraint limits the step size, and the GMRES tolerance must be carefully balanced against the outer Newton tolerance.

To compare these methods fairly, one needs a **computational-time callback** in NKSearch.jl that records the elapsed CPU (or wall-clock) time at each iteration alongside the residual norm. Such a callback is orthogonal to the existing per-iteration logging — it adds a time-stamp column to the CSV output, enabling plots of  $\|F(z)\|$  versus *compute time* rather than iteration count. This would allow questions such as “which method reaches  $\|F\| = 10^{-6}$  in the shortest wall-clock time?” to be answered quantitatively. A preliminary comparison on the Open Kolmogorov Flow system would be particularly instructive, since the higher state-space dimension of OKF (hundreds to thousands of degrees of freedom, versus three for Lorenz) amplifies the cost difference between adjoint-based and adjoint-free methods. The callback would also record the time spent in each computational phase (residual evaluation, gradient computation, GMRES iterations, line search), yielding a detailed performance profile.

## References

- [1] E. N. Lorenz. Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20(2):130–141, 1963.
- [2] P. Cvitanović, R. Artuso, R. Mainieri, G. Tanner, and G. Vattay. *Chaos: Classical and Quantum*. Niels Bohr Institute, Copenhagen, 2016. <http://chaosbook.org/>
- [3] J. Nocedal and S. J. Wright. *Numerical Optimization*, 2nd ed. Springer, New York, 2006.
- [4] J. Sánchez, M. Net, B. García-Archilla, and C. Simó. Newton–Krylov continuation of periodic orbits for Navier–Stokes flows. *Journal of Computational Physics*, 201(1):13–33, 2004.
- [5] D. Viswanath. Recurrent motions within plane Couette turbulence. *Journal of Fluid Mechanics*, 580:339–358, 2007.
- [6] G. J. Chandler and R. R. Kerswell. Invariant recurrent solutions embedded in a turbulent two-dimensional Kolmogorov flow. *Journal of Fluid Mechanics*, 722:554–595, 2013.
- [7] D. C. Liu and J. Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45(1–3):503–528, 1989.